

《算法设计技巧与分析》

第5章 分治

朱友文

zhuyw@nuaa.edu.cn

5.1 引言

- 找出 $A[1, \dots, n]$ 的最大值和最小值
- 直观的方法

```
1.  $x \leftarrow A[1]; y \leftarrow A[1]$   
2. for  $i \leftarrow 2$  to  $n$   
3.     if  $A[i] < x$  then  $x \leftarrow A[i]$   
4.     if  $A[i] > y$  then  $y \leftarrow A[i]$   
5. end for  
6. return  $(x, y)$ 
```

比较次数:
 $2n-2$

5.1 引言

➤ 找出 $A[1, \dots, n]$ 的最大值和最小值

假设 n 是 2 的整数幂

• 分治法:

$$\begin{aligned} mid &\leftarrow \lfloor (low + high) / 2 \rfloor \\ (x_1, y_1) &\leftarrow \text{minmax}(low, mid) \\ (x_2, y_2) &\leftarrow \text{minmax}(mid + 1, high) \\ x &= \min\{x_1, x_2\}; \\ y &= \max\{y_1, y_2\}; \end{aligned}$$

比较次数:
$$C(n) = \begin{cases} 1, & n = 2 \\ 2C(n/2) + 2, & n > 2 \end{cases}$$

5.1 引言

➤ 找出 $A[1, \dots, n]$ 的最大值和最小值

假设 n 是 2 的整数幂

- 分治法的复杂分析(元素比较次数)

$$\begin{aligned} C(n) &= 2C(n/2) + 2 \\ &= 2(2C(n/4) + 2) + 2 \\ &= 4C(n/4) + 4 + 2 \\ &= 4(2C(n/8) + 2) + 4 + 2 \\ &= 8C(n/8) + 8 + 4 + 2 \\ &\vdots \\ &= 2^{k-1} C(2) + \sum_{j=1}^{k-1} 2^j \\ &= (n/2) + 2^k - 2 \\ &= 3n/2 - 2 \end{aligned}$$

5.2 二分搜索

➤ 算法

过程 *binarysearch*(*low* , *high*)

1. **if** *low* > *high* **then return** 0
2. **else**
3. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$
4. **if** $x = A[mid]$ **then return** *mid*
5. **else if** $x < A[mid]$ **then return** *binarysearch*(*low* , *mid* - 1)
6. **else return** *binarysearch*(*mid* + 1 , *high*)
7. **end if**

5.2 二分搜索

➤ 复杂度(最坏): 令 $n = high - low + 1$,
 $C(n)$ 表示最坏情况下的比较次数, 那么

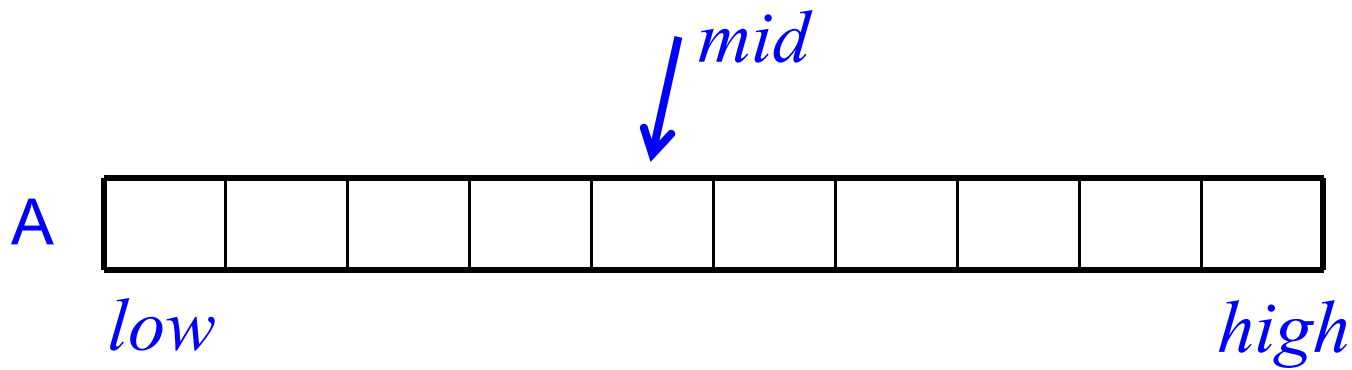
$$C(n) \leq \begin{cases} 1 & \text{若 } n = 1 \\ 1 + C(\lfloor n/2 \rfloor) & \text{若 } n \geq 2 \end{cases}$$

不妨设 $2^{k-1} \leq n < 2^k$, 则

$$\begin{aligned} C(n) &\leq 1 + C(\lfloor n/2 \rfloor) \\ &\leq 2 + C(\lfloor n/4 \rfloor) \\ &\vdots \\ &\leq (k-1) + C(\lfloor n/2^{k-1} \rfloor) \\ &= (k-1) + 1 \\ &= k \end{aligned}$$

所以, $C(n) = O(\log n)$

- 把含有 n 个元素的数组 $X=[x_1, x_2, \dots, x_n]$ 最开始的 k 个元素($0 \leq k < n$)依次移到数组的末尾, 得到的结果为 $X'=[x_{k+1}, x_{k+2}, \dots, x_n, x_1, x_2, \dots, x_k]$, 称为数组 X 的 k -旋转, 数组 X 本身称为 X 的0-旋转
- 现有一个包含 n 个互不相同的元素的有序数组, 其中元素按照升序排列, 该有序数组的某个旋转存储在 $A[1,2,\dots,n]$ 中, 设计一个 $O(\log n)$ 算法, 求出 $A[1,2,\dots,n]$ 中的最大元素。例如 $\{5, 7, 9, 1, 3\}$ 为有序数组 $\{1, 3, 5, 7, 9\}$ 的一个旋转, 该数组的最大元素为9.



如果 $A[low] < A[mid]$ ，则最大值在 $A[mid...high]$ 中；
否则，最大值在 $A[low...mid-1]$ 中

- 现有一个包含 n 个互不相同的元素的有序数组，其中元素按照升序排列，该有序数组的某个旋转存储在 $A[1,2,...,n]$ 中，设计一个 $O(\log n)$ 算法，求出 $A[1,2,...,n]$ 中的最大元素。例如 $\{5, 7, 9, 1, 3\}$ 为有序数组 $\{1, 3, 5, 7, 9\}$ 的一个旋转，该数组的最大元素为 9。

SearchMax(*A*, *low*, *high*) // *A*[*low*...*high*]是一个增序序列的旋转，从中搜索出最大值

```
{  
    if(low==high) return A[low];  
    k=[(low + high)/2];  
    if(A[low]<A[k]) SearchMax(A, k, high);  
        // A[low]<A[k]时，最大值只可能在 A[k...high]中，  
        //且 A[k...high]是某增序序列的旋转  
    else SearchMax(A, low, k-1); // A[low]>A[k]时，最大值只可能在 A[low...k-1]中，  
        //且 A[low...k-1]是某增序序列的旋转  
}
```

5.3 合并排序

➤ 算法 过程 *mergesort*(*low*, *high*)

1. **if** *low* < *high* **then**
2. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$
3. *mergesort*(*A*, *low*, *mid*)
4. *mergesort*(*A*, *mid* + 1, *high*)
5. MERGE(*A*, *low*, *mid*, *high*)
6. **end if**

8	4	3	1	6	2	9	7
1	2	3	4	6	7	8	9

1

8	4	3	1
1	3	4	8

6	2	9	7
2	6	7	9

2

7

8

8	4
4	8

3	1
1	3

6	2
2	6

9	7
7	9

3

4

5

6

8
8

4
4

3
3

1
1

6
6

2
2

9
9

7
7

5.3 合并排序

- 调用的顺序：前序遍历
- 处理的顺序：后序遍历

5.3 合并排序

➤ 复杂度分析(比较次数、赋值次数):

- 当 $n=2^k$ 时, 最坏情况下比较次数

$$C(n) = \begin{cases} 0 & \text{若 } n = 1 \\ 2C(n/2) + n - 1 & \text{若 } n \geq 2 \end{cases}$$

$$C(n) = O(n \log n)$$

$$\begin{aligned} C(n) &= 2C(n/2) + n - 1 \\ &= 2(2C(n/2^2) + n/2 - 1) + n - 1 \\ &= 2^2 C(n/2^2) + n - 2 + n - 1 \\ &= 2^2 C(n/2^2) + 2n - 2 - 1 \\ &\vdots \\ &= 2^k C(n/2^k) + kn - 2^{k-1} - 2^{k-2} - \cdots - 2 - 1 \\ &= 2^k C(1) + kn - \sum_{j=0}^{k-1} 2^j \\ &= 2^k \times 0 + kn - (2^k - 1) \quad (\text{等式 2.10}) \\ &= kn - 2^k + 1 \\ &= n \log n - n + 1 \end{aligned}$$

5.3 合并排序

➤ 复杂度分析(比较次数、赋值次数)：

- 赋值次数： $2n * \text{递归层数} = O(n \log n)$

➤ 空间复杂度分析： $\Theta(n)$

5.4 分治范式

- 分治算法的好处：
- 递归算法和迭代算法的转换

5.4 分治范式

➤ 主要思路:

- 划分步骤: 分成 $p \geq 1$ 部分
- 治理步骤: 递归过程 $\rightarrow$ 阈值 n_0
- 组合步骤: 将 p 个部分解, 组合成原始问题的解

5.4 分治范式

➤ 分治算法

- (1) 如果问题 I 的规模很小，直接求解，否则做下一步；
- (2) 把问题 I 划分为 p 个子问题 $I_1, I_2, \dots, I_p$ ；
- (3) 对每个子问题，递归调用算法，得到子问题的解；
- (4) 组合 p 个子问题的解，返回 I 的解

5.5 寻找第 k 小元素

- 给定已排好序(非降序)的数组 $A[1 \dots n]$,中项等于其“中间”元素, 即为第 $\lceil n/2 \rceil$ 个最小元素
- 寻找中项的一个直接方法: 先排序, 后取中项。该方法的时间复杂度至少为 $\Omega(n \log n)$ 。能否找到更为高效的方法?
- 寻找中项是寻找第 k 小元素的一个特例。如果能解决寻找第 k 小元素的问题, 那么当 $k = \lceil n/2 \rceil$ 时, 解决的就是寻找中项问题
- 回顾二分搜索: 以中间元素为基准排除部分元素, 不断减小问题规模

5.5 寻找第 k 小元素

➤ 数组 $A[1,2,\dots,n]$ 中第 k 小元素的特点(充要条件):

对任意 $x \in A$, 定义三个集合

$$A_1 = \{a \mid a < x\}, \quad A_2 = \{a \mid a = x\},$$

$$A_3 = \{a \mid a > x\}$$

那么, x 为第 k 小元素当且仅当 $|A_1| < k$ 且 $|A_1| + |A_2| \geq k$

5.5 寻找第 k 小元素

➤ 解决思路:

(1) 找出一个候选值 m

令 $q = \lfloor p/5 \rfloor$ 。将 A 分成 q 组，每组 5 个元素。如果 5 不整除 p ，则排除剩余的元素。

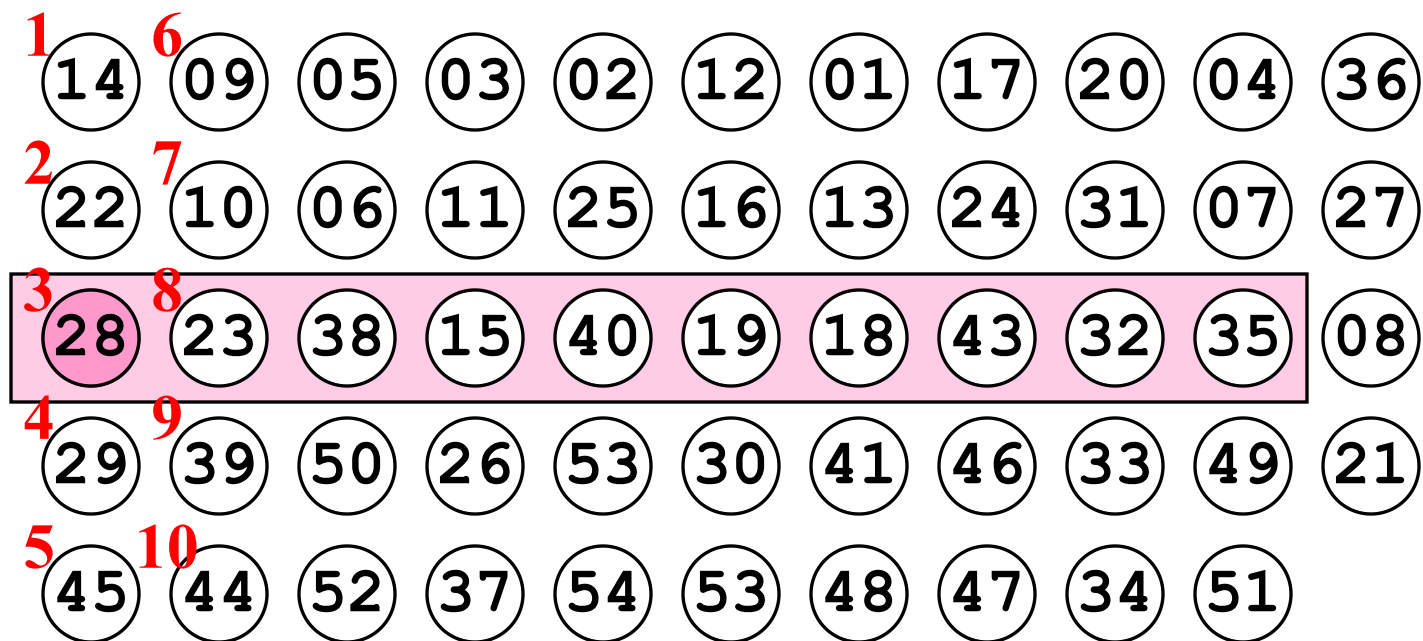
将 q 组中的每一组单独排序，找出中项。所有中项的集合为 M 。

然后，递归调用自身选出 q 个中项的中项

令 $q = \lfloor p/5 \rfloor$ 。将 A 分成 q 组，每组 5 个元素。如果 5 不整除 p ，则排除剩余的元素。

将 q 组中的每一组单独排序，找出中项。所有中项的集合为 M 。

然后，递归调用自身选出 q 个中项的中项



5.5 寻找第 k 小元素

➤ 递归子函数 **SELECT(A, *low*, *high*, *k*)**

调用

Main()

{

.....

//已有数组A[1... n], 计算数组A的第 k 小元素;

SELECT(A, 1, n , k);

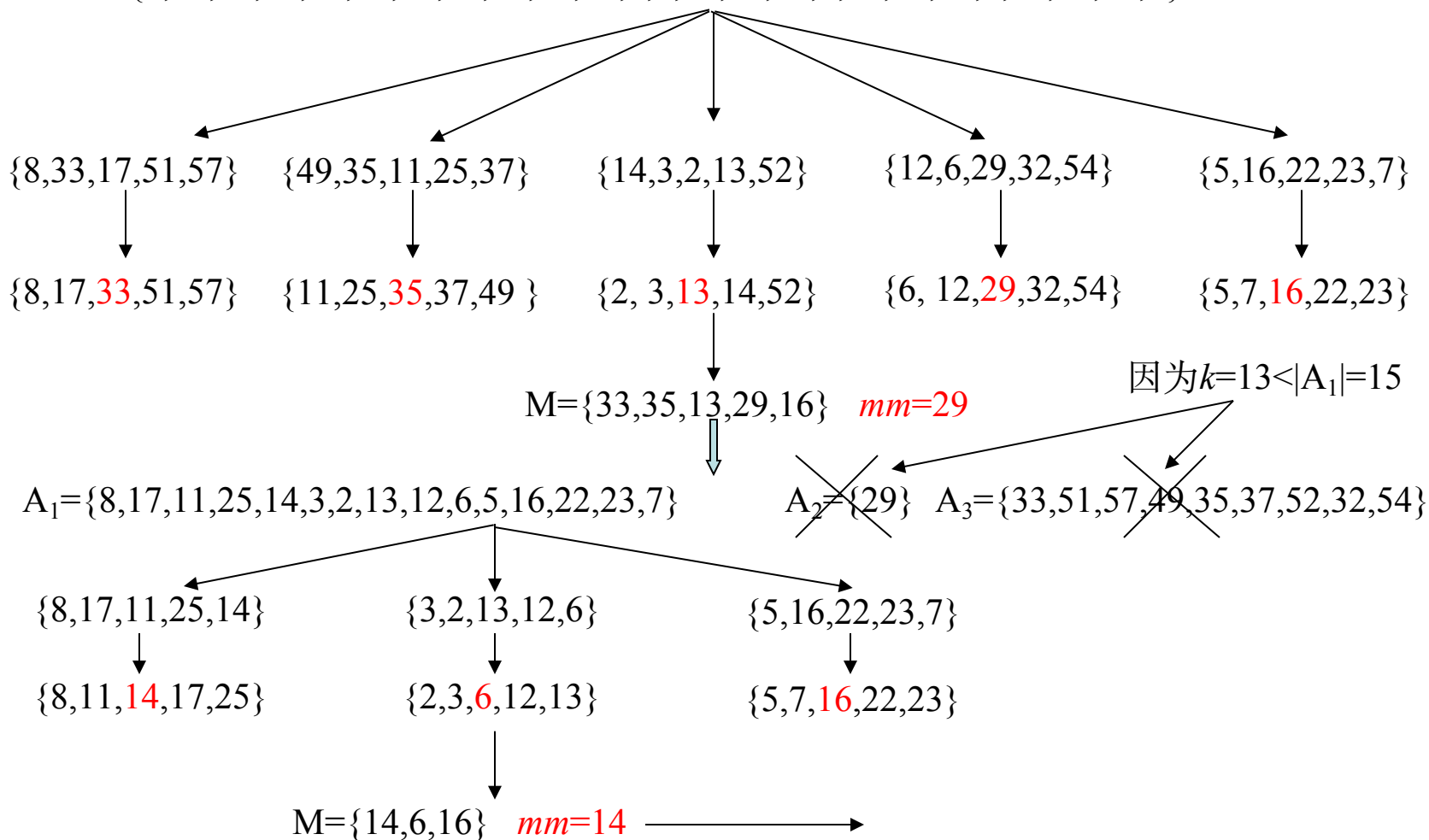
.....

}

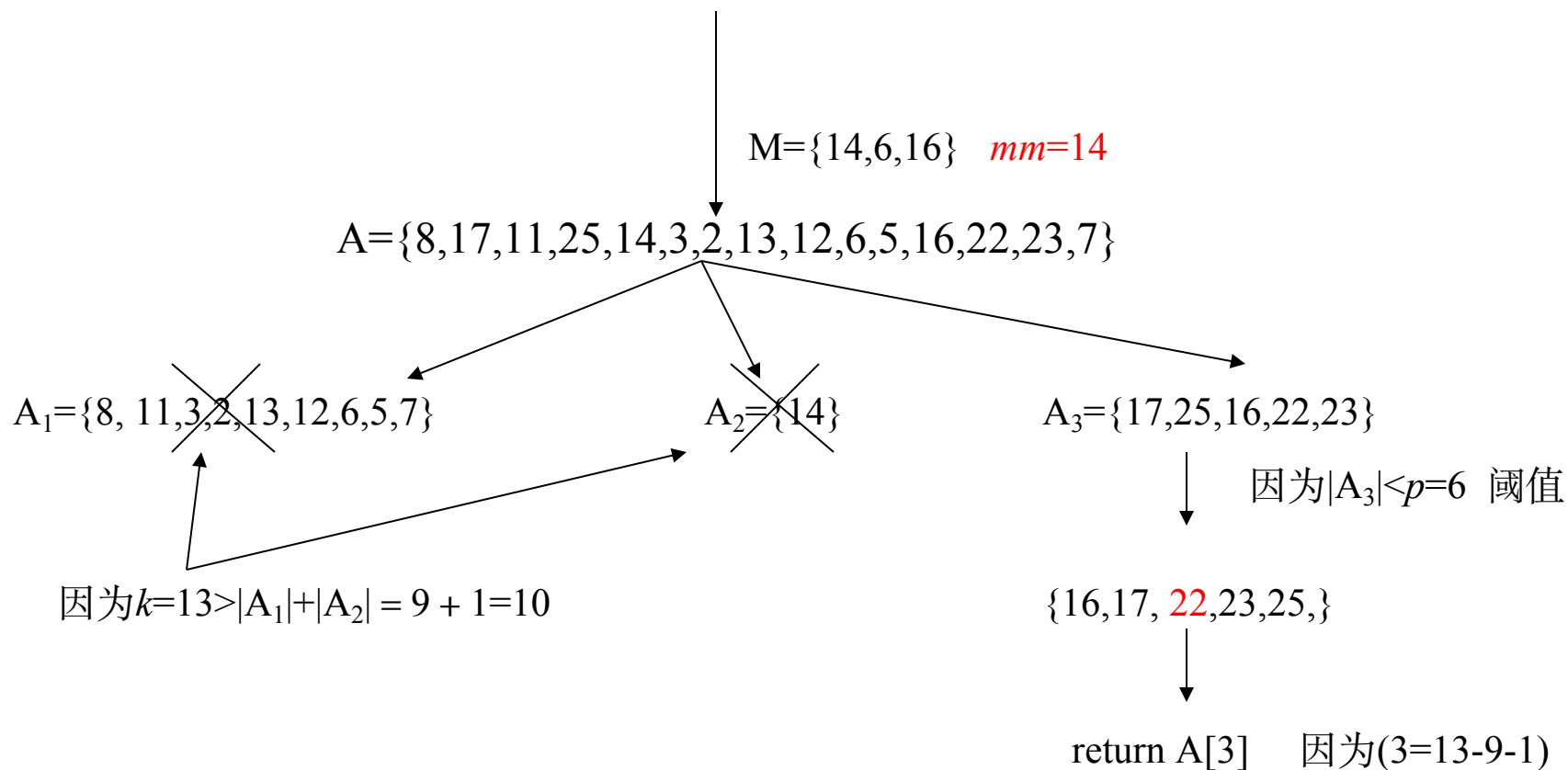
5.5 寻找第 k 小元素

例5.1: 为方便演示, 设阈值为6。现要寻找下面数组A中的第13小元素:

$A = \{8, 33, 17, 51, 57, 49, 35, 11, 25, 37, 14, 3, 2, 13, 52, 12, 6, 29, 32, 54, 5, 16, 22, 23, 7\}$



5.5 寻找第 k 小元素



思考一个问题

➤ 如何实现，将数组A划分三组 A_1 , A_2 和 A_3 ?

6. 将 $A[low \cdots high]$ 分成三组

$$A_1 = \{ a \mid a < mm \}$$

$$A_2 = \{ a \mid a = mm \}$$

$$A_3 = \{ a \mid a > mm \}$$

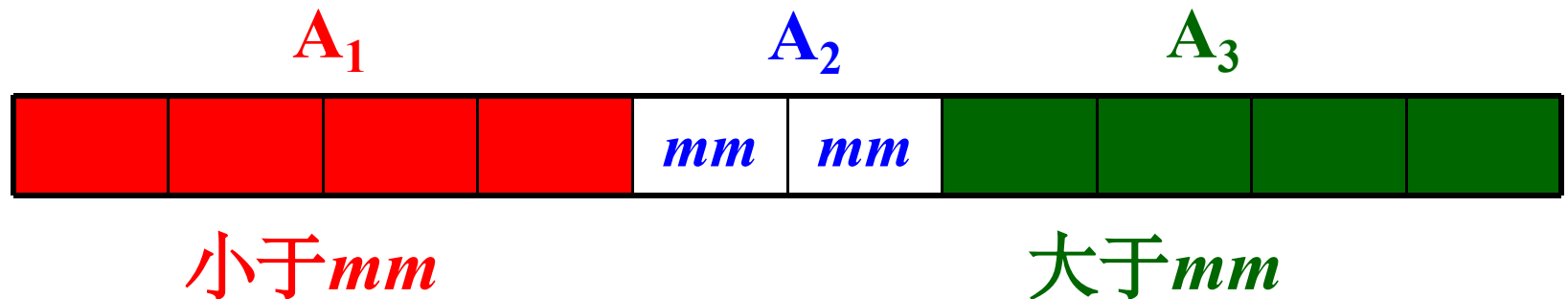
直接的方法：使用三个辅助 A_1 , A_2 和 A_3

空间开销太高!

思考一个问题

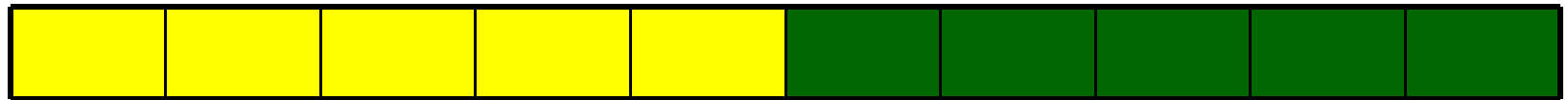
- 如何不需要辅助数组，分成 A_1 , A_2 和 A_3 ?

将 $A[1, \dots, n]$ 分成三块，使得小于 mm 的在左边，等于 mm 的在中间，大于 mm 的右边，那么



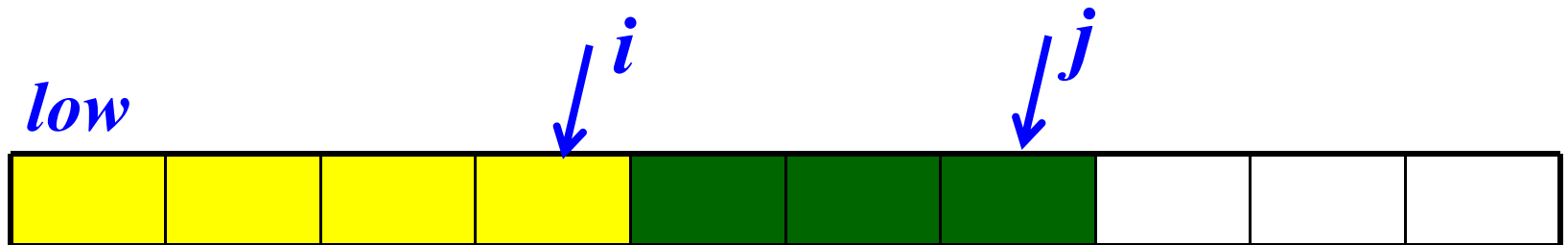
思考一个问题

- 先考虑一种更简单的情况,
- 思路: 先分成两块, 再分成三块



小于等于 mm

大于 mm



小于等于 mm

大于 mm

算法怎么实现?

```
i=low-1;  
for(j=low; j ≤ high; j++){  
    if(A[j]≤mm) {  
        i++;  
        if(j≠i) 交换A[i]和A[j];  
    }  
}
```

```

i=low-1;
for(j=low; j ≤ high; j++){
    if(A[j] ≤ mm) {        i++;
                            if(j ≠ i) 交换A[i]和A[j];
                            }
    }
t = i+1;
for(j=i; j ≥ low; j--){
    if(A[j] == mm){
        t--;
        交换A[t]和A[j];
    }
}

```

这时，集合 A_1 , A_2 和 A_3 分别是？他们的元素个数分别是？用 $i, j, t, low, high$ 表示出它们

(2) 数组A中元素可能相等的情况

集合	对应元素	元素个数
A_1		
A_2		
A_3	$A[i+1, \dots, high]$	$high - i$

思考一个问题

- 一个for循环，直接分成三块，怎么做？

5.5 寻找第 k 小元素

➤ 算法复杂度分析：

- 回到算法，看看每一步复杂度
- 假设 n 个元素的数组的运行时间是 $T(n)$

5.5 寻找第 k 小元素

➤ 递归子函数 **SELECT**($A, low, high, k$)

1. $p \leftarrow high - low + 1$

2. **if** $p < 44$ **then** 将 A 排序 **return** ($A[k]$)

$O(1)$

3. 令 $q = \lfloor p/5 \rfloor$ 。将 A 分成 q 组，每组 5 个元素。如果 5 不整除 p ，则排除剩余的元素。

4. 将 q 组中的每一组单独排序，找出中项。所有中项的集合为 M 。

5. $mm \leftarrow select(M, 1, q, \lceil q/2 \rceil)$ | mm 为中项集合的中项 |

6. 将 $A[low \cdots high]$ 分成三组

$A_1 = \{a \mid a < mm\}$

$A_2 = \{a \mid a = mm\}$

$A_3 = \{a \mid a > mm\}$

7. **case**

$|A_1| \geq k$: **return** $select(A_1, 1, |A_1|, k)$

$|A_1| + |A_2| \geq k$: **return** mm

$|A_1| + |A_2| < k$: **return** $select(A_3, 1, |A_3|, k - |A_1| - |A_2|)$

8. **end case**

$O(n)$

???

$O(1)$

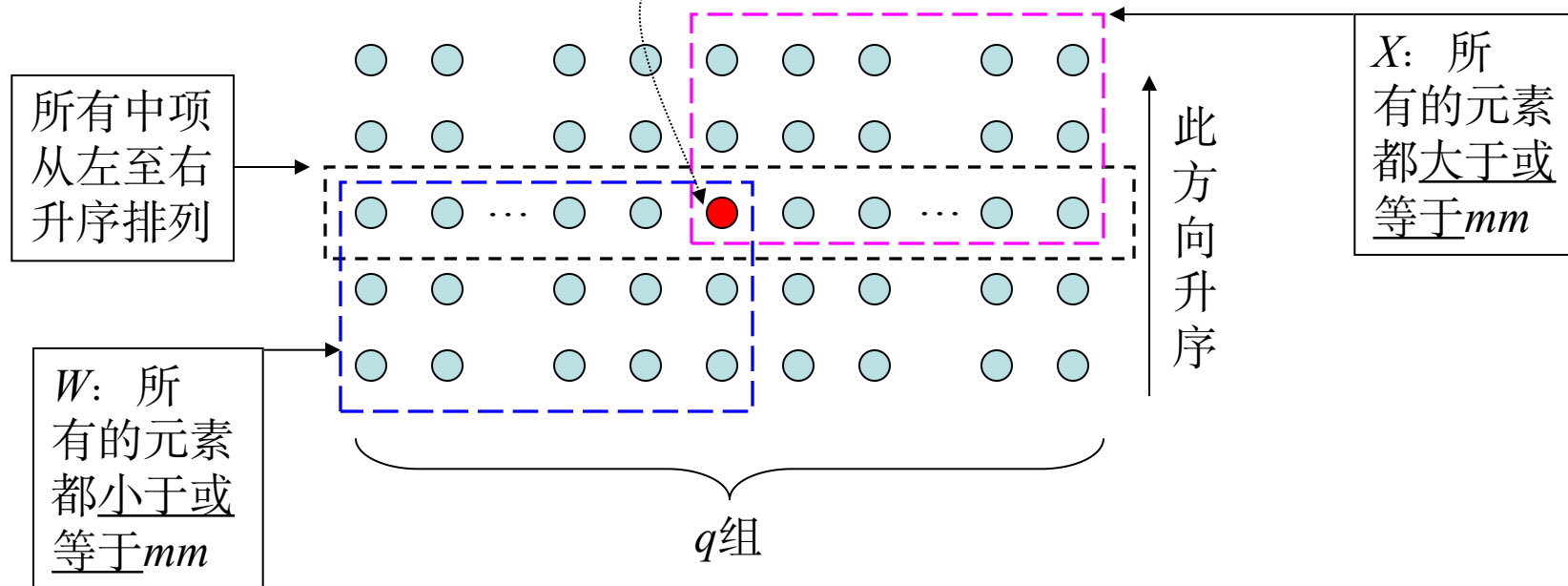
5.5 寻找第 k 小元素

➤ 算法分析：

➤ $|A_1|$, $|A_2|$, $|A_3|$ 的上界是？

5.5 寻找第 k 小元素

mm , 中项的中项



- ◆ A_1' 表示 A 中小于或等于 mm 的元素集, A_1 是 A 中严格小于 mm 的集合, A_3 是 A 中严格大于 mm 的集合
- ◆ 那么 A_1' 至少与 W 同样大(为什么?), 所以 $|A_1'| \geq 3 \lceil \lfloor n/5 \rfloor / 2 \rceil \geq 3/2 \lfloor n/5 \rfloor$,
- ◆ 所以, $|A_3| = |A - A_1'| \leq n - 3/2 \lfloor n/5 \rfloor \leq n - 3/2 ((n-4)/5) = 0.7n + 1.2$
- ◆ 由对称性, 我们有 $|A_1| \leq 0.7n + 1.2$

5.5 寻找第 k 小元素

► 算法分析:

$$T(n) \leq \begin{cases} c & \text{若 } n < 44 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + cn & \text{若 } n \geq 44 \end{cases}$$

根据定理 2.7, 递推式的解是 $T(n) = \Theta(n)$

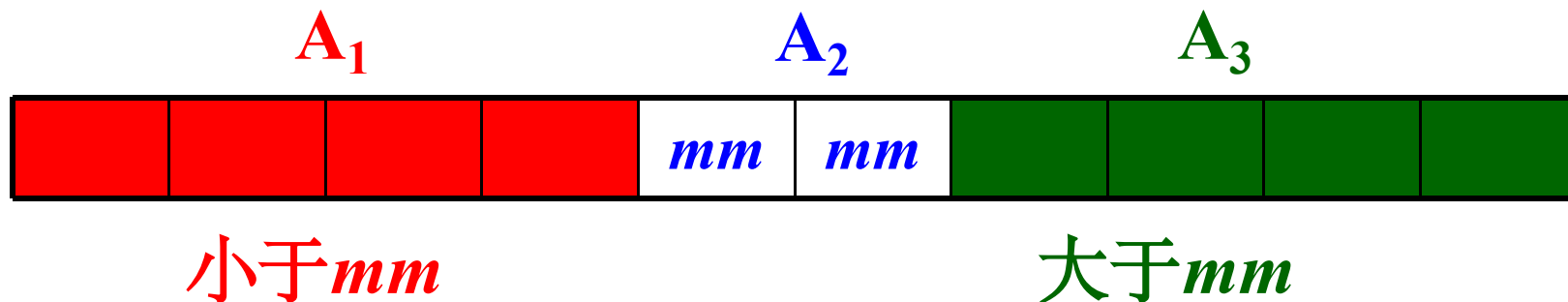
$$T(n) \leq \frac{cn}{1 - 1/5 - 3/4} = 20cn$$

5.5 寻找第 k 小元素

- 特别地：令 $k=\lceil n/2 \rceil$ ，则 n 个元素的中值可以在 $O(n)$ 的时间内找出

再看一看 A_1 , A_2 和 A_3

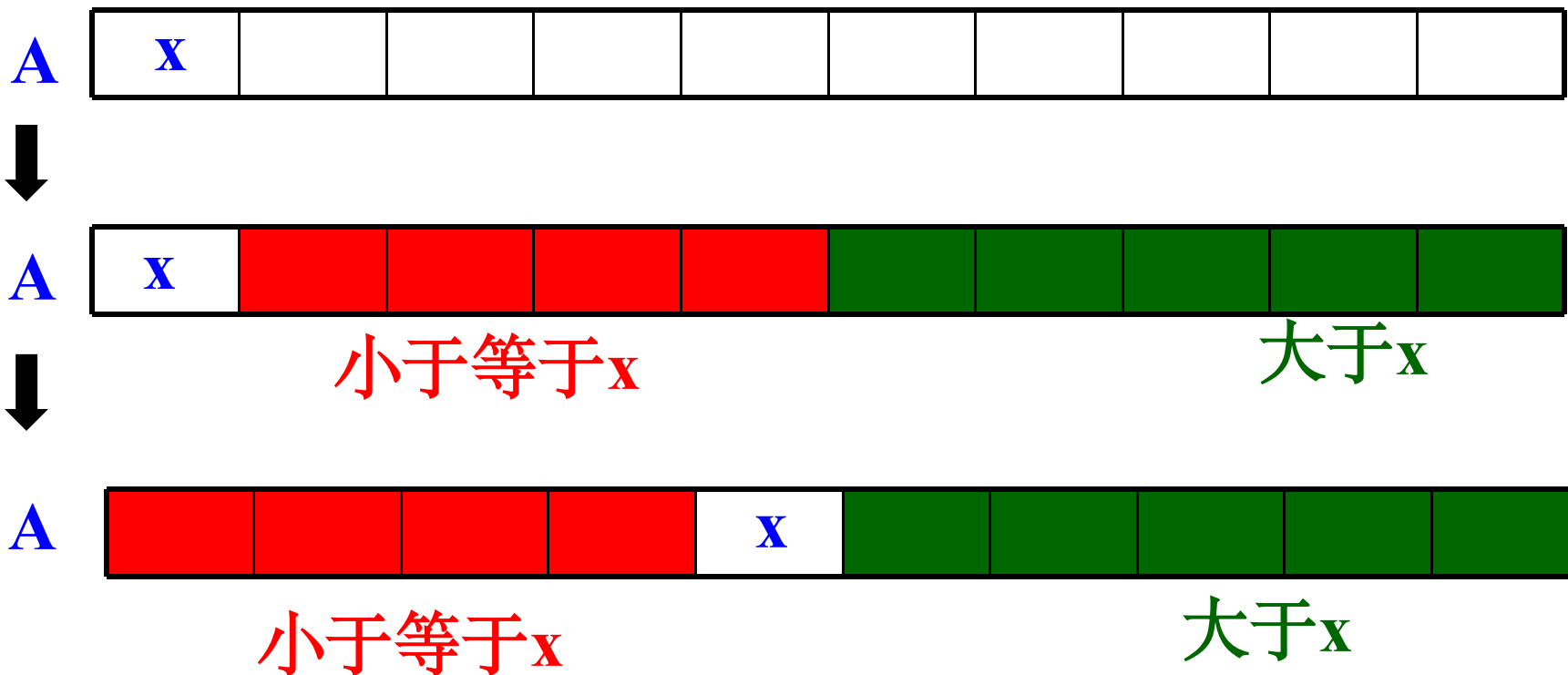
将 $A[\text{low}, \dots, \text{high}]$ 分成三块, 使得小于 mm 的在左边, 等于 mm 的在中间, 大于 mm 的右边, 那么



- 数组 A 被处理后, 具有什么特点?
- 执行后, mm 处于什么状态?

5.6 快速排序

➤ 围绕 x 的划分或拆分



x 称为主元或拆分元素。

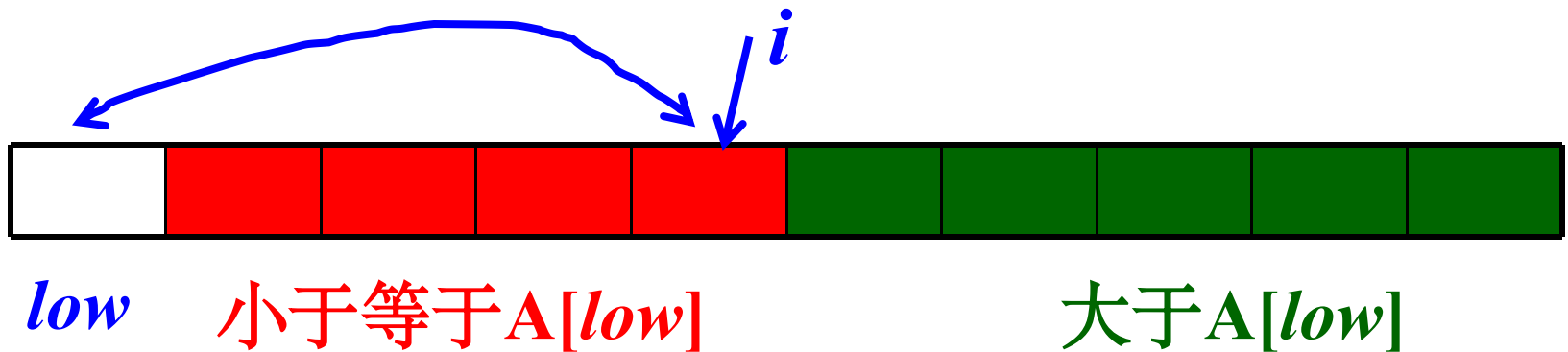
5.6 快速排序

➤ 划分算法

算法5.5 SPLIT(A , low , $high$)

输入: 数组 $A[low \dots high]$

输出: 围绕 $x = A[low]$ 的拆分, x 的新位置 w



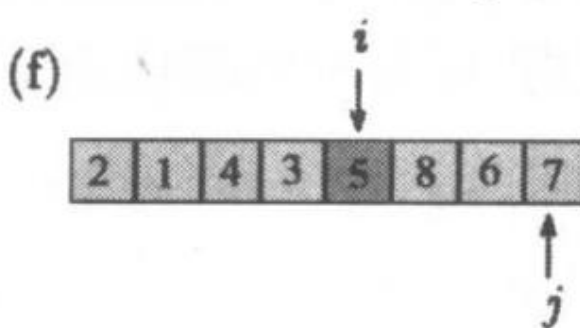
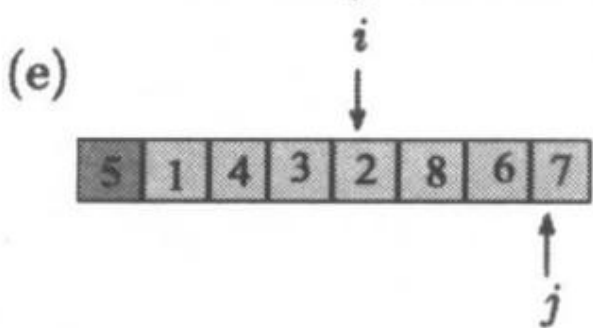
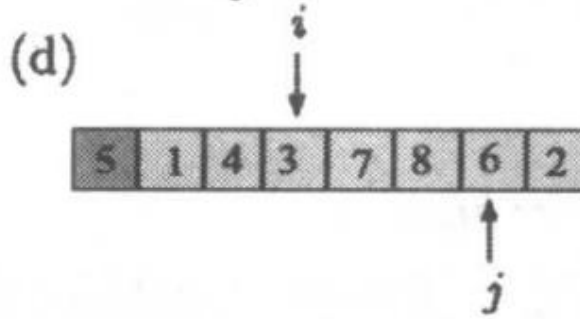
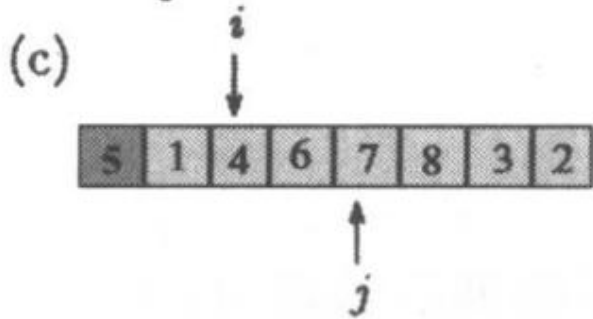
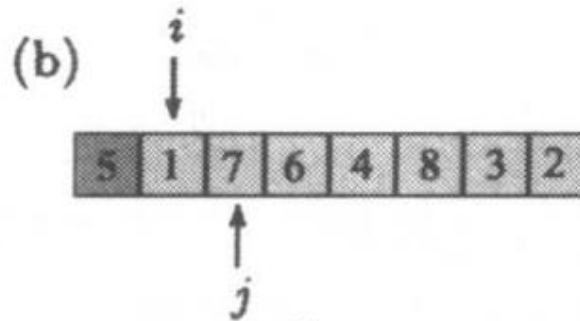
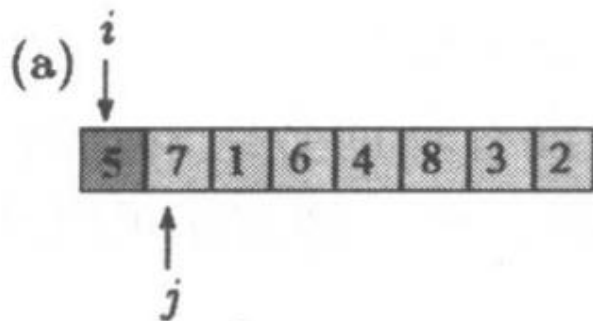
5.6 快速排序

➤ 划分算法

1. $i \leftarrow low$
2. $x \leftarrow A[low]$
3. **for** $j \leftarrow low + 1$ **to** $high$
4. **if** $A[j] \leq x$ **then**
5. $i \leftarrow i + 1$
6. **if** $i \neq j$ **then** 互换 $A[i]$ 和 $A[j]$
7. **end if**
8. **end for**
9. 互换 $A[low]$ 和 $A[i]$
10. $w \leftarrow i$
11. **return** A 和 w

5.6 快速排序

➤ 划分算法:



5.6 快速排序

➤ 划分算法：复杂度分析

基本运算：比较

比较次数： $n-1$

5.6 快速排序

► 排序算法：子函数

过程 $quicksort(A, low, high)$

1. **if** $low < high$ **then**
2. $SPLIT(A[low \cdots high], w)$ $\{w \text{ 为 } A[low] \text{ 的新位置}\}$
3. $quicksort(A, low, w - 1)$
4. $quicksort(A, w + 1, high)$
5. **end if**

5.6 快速排序

➤ 算法复杂度分析

用 $C(n)$ 表示对一个 n 个元素的数组进行快速排序所需要的总的比较次数。那么

$$C(n) = ?$$

$$C(1) = 0;$$

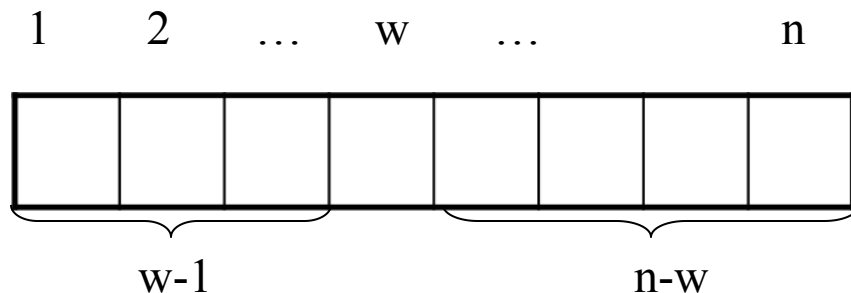
$$n > 1 \text{ 时, } C(n) = (n-1) + C(X_{\text{左}}) + C(n-1-X_{\text{左}})$$

5.6 快速排序

➤ 算法复杂度分析

平均情况

我们用 $C(n)$ 表示对一个 n 个元素的数组进行快速排序所需要的总的比较次数。



因此，我们有：

$$C(n) = (n-1) + \frac{1}{n} \sum_{w=1}^n (C(w-1) + C(n-w))$$

$$\therefore \sum_{w=1}^n C(n-w) = C(n-1) + C(n-2) + \cdots + C(0) = \sum_{w=1}^n C(w-1)$$

$$\therefore C(n) = (n-1) + \frac{2}{n} \sum_{w=1}^n C(w-1)$$

$$n \cdot C(n) = n(n-1) + 2 \sum_{w=1}^n C(w-1) \dots (a)$$

$\Downarrow$ n-1替换n

$$(n-1)C(n-1) = (n-1)(n-2) + 2 \sum_{w=1}^{n-1} C(w-1) \dots (b)$$

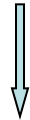
(a)-(b), 并适当变换



$$\frac{C(n)}{n+1} = \frac{C(n-1)}{n} + \frac{2(n-1)}{n(n+1)}$$

$$\text{令 } D(n) = \frac{C(n)}{n+1} \quad \Downarrow$$

$$D(n) = D(n-1) + \frac{2(n-1)}{n(n+1)}, D(1) = 0$$



$$D(n) = 2 \sum_{j=1}^n \frac{j-1}{j(j+1)} = 2 \sum_{j=1}^n \frac{2}{(j+1)} - 2 \sum_{j=1}^n \frac{1}{j}$$

$$= 4 \sum_{j=2}^{n+1} \frac{1}{j} - 2 \sum_{j=1}^n \frac{1}{j} = 2 \sum_{j=1}^n \frac{1}{j} - \frac{4n}{n+1} = \cdot \cdot (\log n)$$

$$\therefore C(n) = (n+1)D(n) = \cdot \cdot (n \log n)$$

5.6 快速排序

➤ 算法复杂度分析

最坏情况

1	2	3	4	6	7	8	9
---	---	---	---	---	---	---	---

9	8	7	6	4	3	2	1
---	---	---	---	---	---	---	---

最坏情形发生的原因？

$$n > 1 \text{ 时, } C(n) = (n-1) + C(X_{\text{左}}) + C(n-1-X_{\text{左}})$$

思考一个小问题

► 复数的乘法:

假设做一次实数乘法需要20秒，实数加/减法需要2秒，那么，计算下面等式中 x 和 y 最少需要多少秒

$$(a+bi)(c+di)=x+yi$$

显然： $x=ac-bd$ ， $y=ad+bc$

$$y=ad+bc = (a+b)(c+d) - ac - bd$$

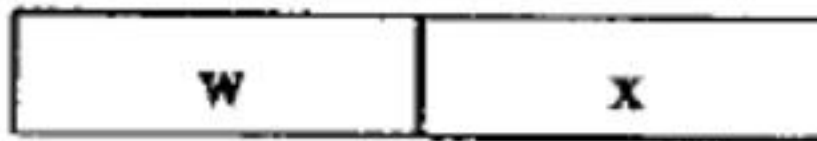
所以： $(20+20+2)+(20+20+2)=84$



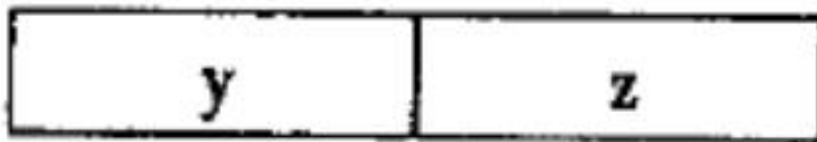
5.8 大整数乘法

► 不定长的大整数相乘

$$u = w2^{n/2} + x$$



$$v = y2^{n/2} + z$$



$$\begin{aligned} u * v &= (w2^{n/2} + x)(y2^{n/2} + z) \\ &= wy2^n + (wz + xy)2^{n/2} + xz \end{aligned}$$

5.8 大整数乘法

$$u * v = wy2^n + (wz + xy)2^{n/2} + xz$$

$$T(n) = \begin{cases} d & \text{若 } n = 1 \\ 4T(n/2) + bn & \text{若 } n > 1 \end{cases}$$

$$T(n) = \Theta(n^2)$$

5.8 大整数乘法

$$u * v = wy2^n + (wz + xy)2^{n/2} + xz$$

$$wz + xy = (w + x)(y + z) - wy - xz$$

$$T(n) = \begin{cases} d & \text{若 } n = 1 \\ 3T(n/2) + bn & \text{若 } n > 1 \end{cases}$$

$$T(n) = \Theta(n^{\log_3}) = \Theta(n^{1.59})$$

5.9 矩阵乘法

➤ 传统的方法计算 $C=AB$

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

➤ 复杂度：乘法，加法

5.9 矩阵乘法

➤ STRASSEN方法计算 $C=AB$

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

$$C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

5.9 矩阵乘法

$$C = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$

$$d_1 = (a_{11} + a_{22})(b_{11} + b_{22}) \qquad d_2 = (a_{21} + a_{22})b_{11}$$

$$d_3 = a_{11}(b_{12} - b_{22}) \qquad d_4 = a_{22}(b_{21} - b_{11})$$

$$d_5 = (a_{11} + a_{12})b_{22} \qquad d_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$d_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

$$C = \begin{pmatrix} d_1 + d_4 - d_5 + d_7 & d_3 + d_5 \\ d_2 + d_4 & d_1 + d_3 - d_2 + d_6 \end{pmatrix}$$

5.9 矩阵乘法

➤ 以 a 和 m 分别表示加法和乘法耗费的时间

$$T(n) = \begin{cases} m & \text{若 } n = 1 \\ 7T(n/2) + 18(n/2)^2 a & \text{若 } n \geq 2 \end{cases}$$

$$T(n) = \Theta(n^{\log_2 7}) = O(n^{2.81})$$

课后作业

作业： 5.29

5.29 设 $A[1 \cdots n]$ 和 $B[1 \cdots n]$ 是两个已按升序排列的互不相同的整数所组成的数组，给出一个有效的算法找出在 A 和 B 中 $2n$ 个元素的中项。你的算法的运行时间是多少？

- 4月16日（周四）上课前，交作业。